rate of development, we won't be able to ship the product until December!"

Actually, you think it's more like March, but you don't say anything.

"December!" BB roars with such derision that people duck their heads as though he were pointing an assault rifle at them. "December is absolutely out of the question. Team leaders, I want new estimates on my desk in the morning. I am hereby mandating 65-hour workweeks until this project is complete. And it better be complete by November 1."

As he leaves the conference room, he is heard to mutter, "Empowerment—Bah!"

* * *

Your boss is bald; his points are mounted on BB's wall. The fluorescent lights reflecting off his pate momentarily dazzle you.

"Do you have anything to drink?" he asks. Having just finished your last bottle of Boone's Farm, you pull a bottle of Thunderbird from your bookshelf and pour it into his coffee mug. "What's it going to take to get this project done?" he asks.

"We need to freeze the requirements, analyze them, design them, and then implement them." You say callously.

"By November 1?" your boss exclaims incredulously. "No way! Just get back to coding the damned thing." He storms out, scratching his vacant head.

A few days later, you find that your boss has been transferred to the corporate research division. Turnover has skyrocketed. Customers, informed at the last minute that their orders cannot be fulfilled on time, have begun to cancel their orders. Marketing is reevaluating whether or not this product aligns with the overall goals of the company, etc., etc. Memos fly, heads roll, policies change, and things are, overall, pretty grim.

Finally, by March, after far too many 65-hour weeks, a very shaky version of the software is ready. In the field, bug discovery rates are high, and the technical support staff are at their wit's end trying to cope with the complaints and demands of the irate customers. Nobody is happy.

In April, BB decides to buy his way out of the problem by licensing a product produced by Rupert industries and redistributing it. The customers are mollified, the marketing folks are smug, and you are laid off.

*In March, there is a major demonstration of the system to the board of directors. The system is very limited and is not yet in a form good enough to take to the trade show, but progress is steady, and the board is reasonably impressed.*

*The second release goes even smoother than the first. By now, the team has figured out a way to automate Jay's acceptance-test scripts. They have also refactored the design of the system to the point where it is really easy to add new features and change old ones.*

*The second release is done by the end of June and is taken to the trade show. It has less in it than Jay and Russ would have liked, but it does demonstrate the most important features of the system. Though customers at the trade show notice that certain features are missing, overall they are very impressed. You, Russ, and Jay all return from the trade show with smiles on your faces. You all feel as though this project is a winner.*

*Indeed, many months later you are contacted by Rufus, Inc. They had been working on a system like this for their internal operations. They have cancelled the development of that system after a death-march project and are negotiating to license your technology for their environment.*

*Indeed, things are looking up!*

# APPENDIX D

# The Source Code Is the Design

*I can still remember where I was when I had the insight that eventually led to the following article. In the summer of 1986, I was working a temporary consulting assignment at the China Lake Naval Weapons Center in California. While there, I took the opportunity to attend a panel discussion on Ada. At one point, someone in the audience asked the typical question, "Are software developers engineers?" I don't remember the actual answer, but I do recall that it didn't really seem to address the question. So I sat back and started thinking about how I would answer such a question. I am not sure how, but something in the ensuing discussion caused me to recall an article I had read in* Datamation *magazine almost 10 years before that. That article had been a rationale for why engineers needed to be good writers (I think that is what it was about—it has been a long time), but the key point I got from the article was the author's contention that the end result of an engineering process was a document. In other words, engineers produced documents, not things. Other people took those documents and produced things. So, my wandering mind asked the question, "Out of all the documentation that software projects normally generate, was there anything that could truly be considered an engineering document?" The answer that came to me was, "Yes, there was such a document, and only one—the source code."*

*Looking at the source code as an engineering document—a design—turned my view of my chosen profession upside down. It changed the way I looked at everything. Also, the more I thought about it, the more I felt that it explained an awful lot of the problems that software projects typically encountered. Or rather, I felt that the fact that most people did not understand this distinction, or actively rejected it, explained a lot of things. Several more years went by before the opportunity presented itself for me to make my argument publicly. An article about software design in* The C++ Journal *prompted me to write a letter to the editor about the topic. After an exchange of letters, Livleen Singh, the editor, agreed to publish my thoughts on the topic as an article. What follows is the result.*

—Jack Reeves, December, 22, 2001

## What Is Software Design?

© *Jack W. Reeves, 1992*

Object-oriented techniques, C++ in particular, seem to be taking the software world by storm. Numerous articles and books have appeared describing how to apply the new techniques. In general, the questions of whether O-O

techniques are just hype have been replaced by questions of how to get the benefits with the least amount of pain. Object-oriented techniques have been around for some time, but this exploding popularity seems a bit unusual. Why the sudden interest? All kinds of explanations have been offered. In truth, there is probably no single reason. Probably, a combination of factors has finally reached critical mass, and things are taking off. Nevertheless, it seems that C++ itself is a major factor in this latest phase of the software revolution. Again, there are probably a number of reasons why, but I want to suggest an answer from a slightly different perspective: C++ has become popular because it makes it easier to design software and program at the same time.

If that comment seems a bit unusual, it is deliberate. What I want to do in this article is take a look at the relationship between programming and software design. For almost 10 years, I have felt that the software industry collectively misses a subtle point about the difference between developing a software design and what a software design really is. I think there is a profound lesson in the growing popularity of C++ about what we can do to become better software engineers, if only we see it. This lesson is that programming is not about building software; programming is about designing software.

Years ago, I was attending a seminar where the question came up of whether software development is an engineering discipline or not. While I do not remember the resulting discussion, I do remember how it catalyzed my own thinking that the software industry has created some false parallels with hardware engineering while missing some perfectly valid parallels. In essence, I concluded that we are not software engineers because we do not realize what a software design really is. I am even more convinced of that today.

The final goal of any engineering activity is some type of documentation. When a design effort is complete, the design documentation is turned over to the manufacturing team. This is a completely different group with completely different skills from the design team. If the design documents truly represent a complete design, the manufacturing team can proceed to build the product. In fact, they can proceed to build lots of the product, all without any further intervention from the designers. After reviewing the software development life cycle as I understood it, I concluded that the only software documentation that actually seems to satisfy the criteria of an engineering design is the source-code listings.

There are probably enough arguments both for and against this premise to fill numerous articles. This article assumes that final source code is the real software design and then examines some of the consequences of that assumption. I may not be able to prove that this point of view is correct, but I hope to show that it does explain some of the observed facts of the software industry, including the popularity of C++.

There is one consequence of considering code as software design that completely overwhelms all others. It is so important and so obvious that it is a total blind spot for most software organizations. This is the fact that software is cheap to build. It does not qualify as inexpensive; it is so cheap, it is almost free. If source code is a software design, then actually building software is done by compilers and linkers. We often refer to the process of compiling and linking a complete software system as "doing a build." The capital investment in software construction equipment is low—all it really takes is a computer, an editor, a compiler, and a linker. Once a build environment is available, then actually doing a software build just takes a little time. Compiling a 50,000 line C++ program may seem to take forever, but how long would it take to build a hardware system that had a design of the same complexity as 50,000 lines of C++?

Another consequence of considering source code as software design is the fact that a software design is relatively easy to create, at least in the mechanical sense. Writing (i.e., designing) a typical software module of 50 to 100 lines of code is usually only a couple of day's effort (getting it fully debugged is another story, but more on that later). It is tempting to ask if there is any other engineering discipline that can produce designs of such complexity as software in such a short time, but first we have to figure out how to measure and compare complexity. Nevertheless, it is obvious that software designs get very large rather quickly.

Given that software designs are relatively easy to turn out, and essentially free to build, an unsurprising revelation is that software designs tend to be incredibly large and complex. This may seem obvious, but the magnitude of the problem is often ignored. School projects often end up being several thousand lines of code. There are

software products with 10,000-line designs that are given away by their designers. We have long since passed the point where simple software is of much interest. Typical commercial software products have designs that consist of hundreds of thousands of lines. Many software designs run into the millions. Additionally, software designs are almost always constantly evolving. While the current design may only be a few thousand lines of code, many times that may actually have been written over the life of the product.

While there are certainly examples of hardware designs that are arguably as complex as software designs, note two facts about modern hardware. First—complex hardware engineering efforts are not always as free of bugs as software critics would have us believe. Major microprocessors have been shipped with errors in their logic, bridges have collapsed, dams have broken, airliners have fallen out of the sky, and thousands of automobiles and other consumer products have been recalled—all within recent memory and all the result of design errors. Second—complex hardware designs have correspondingly complex and expensive build phases. As a result, the ability to manufacture such systems limits the number of companies that produce truly complex hardware designs. No such limitations exist for software. There are hundreds of software organizations and thousands of very complex software systems in existence. Both the number and the complexity are growing daily. This means that the software industry is not likely to find solutions to its problems by trying to emulate hardware developers. If anything, as CAD and CAM systems have helped hardware designers to create more and more complex designs, hardware engineering is becoming more and more like software development.

Designing software is an exercise in managing complexity. The complexity exists within the software design itself, within the software organization of the company, and within the industry as a whole. Software design is very similar to systems design. It can span multiple technologies and often involves multiple subdisciplines. Software specifications tend to be fluid, changing rapidly and often, usually while the design process is still going on. Software development teams also tend to be fluid, likewise often changing in the middle of the design process. In many ways, software bears more resemblance to complex social or organic systems than to hardware. All of this makes software design a difficult and error-prone process. None of this is original thinking, but almost 30 years after the software engineering revolution began, software development is still seen as an undisciplined art compared to other engineering professions.

The general consensus is that when real engineers get through with a design, no matter how complex, they are pretty sure it will work. They are also pretty sure it can be built using accepted construction techniques. In order for this to happen, hardware engineers spend a considerable amount of time validating and refining their designs. Consider a bridge design, for example. Before such a design is actually built, the engineers do structural analysis—they build computer models and run simulations, they build scale models and test them in wind tunnels or other ways. In short, the designers do everything they can think of to make sure the design is a good design before it is built. The design of a new airliner is even worse; for those, full-scale prototypes must be built and test flown to validate the design predictions.

It seems obvious to most people that software designs do not go through the same rigorous engineering as hardware designs. However, if we consider source code as design, we see that software designers actually do a considerable amount of validating and refining their designs. Software designers do not call it engineering, however, we call it testing and debugging. Most people do not consider testing and debugging as real "engineering"—certainly not in the software business. The reason has more to do with the refusal of the software industry to accept code as design than with any real engineering difference. Mock-ups, prototypes, and breadboards are actually an accepted part of other engineering disciplines. Software designers do not have or use more formal methods of validating their designs because of the simple economics of the software build cycle.

Revelation number one: it is cheaper and simpler to just build the design and test it than to do anything else. We do not care how many builds we do—they cost next to nothing in terms of time, and the resources used can be completely reclaimed later if we discard the build. Note that testing is not just concerned with getting the current design correct, it is part of the process of refining the design. Hardware engineers of complex systems often build models (or at least they visually render their designs using computer graphics). This allows them to get a "feel" for

the design that is not possible by just reviewing the design itself. Building such a model is both impossible and unnecessary with a software design. We just build the product itself. Even if formal software proofs were as automatic as a compiler, we would still do build/test cycles. Ergo, formal proofs have never been of much practical interest to the software industry.

This is the reality of the software development process today. Ever more complex software designs are being created by an ever-increasing number of people and organizations. These designs will be coded in some programming language and then validated and refined via the build/test cycle. The process is error prone and not particularly rigorous to begin with. The fact that a great many software developers do not want to believe that this is the way it works compounds the problem enormously.

Most current software development processes try to segregate the different phases of software design into separate pigeonholes. The top-level design must be completed and frozen before any code is written. Testing and debugging are necessary just to weed out the construction mistakes. In between are the programmers, the construction workers of the software industry. Many believe that if we could just get programmers to quit "hacking" and "build" the designs as given to them (and in the process, make fewer errors) then software development might mature into a true engineering discipline. This is not likely to happen as long as the process ignores the engineering and economic realities.

For example, no other modern industry would tolerate a rework rate of over 100% in its manufacturing process. A construction worker who cannot build it right the first time, most of the time, is soon out of a job. In software, even the smallest piece of code is likely to be revised or completely rewritten during testing and debugging. We accept this sort of refinement during a creative process like design, not as part of a manufacturing process. No one expects an engineer to create a perfect design the first time. Even if she does, it must still be put through the refinement process just to prove that it is perfect.

If we learn nothing else from Japanese management techniques, we should learn that it is counterproductive to blame the workers for errors in the process. Instead of continuing to force software development to conform to an incorrect process model, we need to revise the process so that it helps rather than hinders efforts to produce better software. This is the litmus test of "software engineering." Engineering is about how you do the process, not about whether the final design document needs a CAD system to produce it.

The overwhelming problem with software development is that everything is part of the design process. Coding is design, testing and debugging are part of design, and what we typically call software design is still part of design. Software may be cheap to build, but it is incredibly expensive to design. Software is so complex that there are plenty of different design aspects and their resulting design views. The problem is that all the different aspects interrelate (just like they do in hardware engineering). It would be nice if top-level designers could ignore the details of module algorithm design. Likewise, it would be nice if programmers did not have to worry about top-level design issues when designing the internal algorithms of a module. Unfortunately, the aspects of one design layer intrude into the others. The choice of algorithms for a given module can be as important to the overall success of the software system as any of the higher level design aspects. There is no hierarchy of importance among the different aspects of a software design. An incorrect design at the lowest module level can be as fatal as a mistake at the highest level. A software design must be complete and correct in all its aspects, or all software builds based on the design will be erroneous.

In order to deal with the complexity, software is designed in layers. When a programmer is worrying about the detailed design of one module, there are probably hundreds of other modules and thousands of other details that he cannot possibly worry about at the same time. For example, there are important aspects of software design that do not fall cleanly into the categories of data structures and algorithms. Ideally, programmers should not have to worry about these other aspects of a design when designing code.

This is not how it works, however, and the reasons start to make sense. The software design is not complete until it has been coded and tested. Testing is a fundamental part of the design validation and refinement process. The high-level structural design is not a complete software design; it is just a structural framework for the detailed

design. We have very limited capabilities for rigorously validating a high-level design. The detailed design will ultimately influence (or should be allowed to influence) the high-level design at least as much as other factors. Refining all the aspects of a design is a process that should be happening throughout the design cycle. If any aspect of the design is frozen out of the refinement process, it is hardly surprising that the final design will be poor or even unworkable.

It would be nice if high-level software design could be a more rigorous engineering process, but the real world of software systems is not rigorous. Software is too complex, and it depends on too many other things. Maybe some hardware does not work quite the way the designers thought it did, or a library routine has an undocumented restriction. These are the kinds of problems that every software project encounters sooner or later. These are the kinds of problems discovered during testing (if we do a good job of testing), for the simple reason that there was no way to discover them earlier. When they are discovered, they force a change in the design. If we are lucky, the design changes are local. More often than not, the changes will ripple through some significant portion of the entire software design (Murphy's law). When part of the effected design cannot change for some reason, then the other parts of the design will have to be weakened to accommodate. This often results is what managers perceive as "hacking," but it is the reality of software development.

For example, I recently worked on a project where a timing dependency was discovered between the internals of module A and another module B. Unfortunately, the internals of module A were hidden behind an abstraction that did not permit any way to incorporate the invocation of module B in its proper sequence. Naturally, by the time the problem was discovered, it was much too late to try to change the abstraction of A. As expected, what happened was an increasingly complex set of "fixes" applied to the internal design of A. Before we finished installing version 1, there was the general feeling that the design was breaking down. Every new fix was likely to break some older fix. This is a normal software development project. Eventually, my colleagues and I argued for a change in the design, but we had to volunteer free overtime in order to get management to agree.

On any software project of typical size, problems like these are guaranteed to come up. Despite all attempts to prevent it, important details will be overlooked. This is the difference between craft and engineering. Experience can lead us in the right direction. This is craft. Experience will only take us so far into uncharted territory. Then we must take what we started with and make it better through a controlled process of refinement. This is engineering.

As just a small point, all programmers know that writing the software design documents after the code instead of before, produces much more accurate documents. The reason is now obvious. The final design, as reflected in code, is the only one refined during the build/test cycle. The probability of the initial design being unchanged during this cycle is inversely related to the number of modules and number of programmers on a project. It rapidly becomes indistinguishable from zero.

In software engineering, we desperately need good design at all levels. In particular, we need good top-level design. The better the early design, the easier detailed design will be. Designers should use anything that helps. Structure charts, Booch diagrams, state tables, PDL, etc.—if it helps, then use it. We must keep in mind, however, that these tools and notations are not a software design. Eventually, we have to create the real software design, and it will be in some programming language. Therefore, we should not be afraid to code our designs as we derive them. We simply must be willing to refine them as necessary.

There is as yet no design notation equally suited for use in both top-level design and detailed design. Ultimately, the design will end up coded in some programming language. This means that top-level design notations have to be translated into the target programming language before detailed design can begin. This translation step takes time and introduces errors. Rather than translate from a notation that may not map cleanly into the programming language of choice, programmers often go back to the requirements and redo the top-level design, coding it as they go. This, too, is part of the reality of software development.

It is probably better to let the original designers write the original code, rather than have someone else translate a language-independent design later. What we need is a unified design notation suitable for all levels of design. In other words, we need a programming language that is also suitable for capturing high-level design concepts.

This is where C++ comes in. C++ is a programming language suitable for real-world projects that is also a more expressive software design language. C++ allows us to directly express high-level information about design components. This makes it easier to produce the design and easier to refine it later. With its stronger type checking, it also helps the process of detecting design errors. This results in a more robust design, in essence a better engineered design.

Ultimately, a software design must be represented in some programming language and then validated and refined via a build/test cycle. Any pretense otherwise is just silliness. Consider what software development tools and techniques have gained popularity. Structured programming was considered a breakthrough in its time. Pascal popularized it and in turn became popular. Object-oriented design is the new rage, and C++ is at the heart of it. Now think about what has not worked. CASE tools? Popular, yes; universal, no. Structure charts? Same thing. Likewise, Warner–Orr diagrams, Booch diagrams, object diagrams, you name it. Each has its strengths, and a single fundamental weakness—it really isn't a software design. In fact, the only software design notation that can be called widespread is PDL, and what does that look like?

This says that the collective subconscious of the software industry instinctively knows that improvements in programming techniques, and real-world programming languages in particular, are overwhelmingly more important than anything else in the software business. It also says that programmers are interested in design. When more expressive programming languages become available, software developers will adopt them.

Also consider how the process of software development is changing. Once upon a time, we had the waterfall process. Now we talk of spiral development and rapid prototyping. While such techniques are often justified with terms like "risk abatement" and "shortened product delivery times," they are really just excuses to start coding earlier in the life cycle. This is good. This allows the build/test cycle to start validating and refining the design earlier. It also means that it is more likely that the software designers who developed the top-level design are still around to do the detailed design.

As noted above, engineering is more about how you do the process than it is about what the final product looks like. We in the software business are close to being engineers, but we need a couple of perceptual changes. Programming and the build/test cycle are central to the process of engineering software. We need to manage them as such. The economics of the build/test cycle, plus the fact that a software system can represent practically anything, makes it very unlikely that we will find any general-purpose methods for validating a software design. We can improve this process, but we cannot escape it.

One final point: the goal of any engineering design project is the production of some documentation. Obviously, the actual design documents are the most important, but they are not the only ones that must be produced. Someone is eventually expected to use the software. It is also likely that the system will have to be modified and enhanced at a later time. This means that auxiliary documentation is as important for a software project as it is for a hardware project. Ignoring for now, user manuals, installation guides, and other documents not directly associated with the design process, there are still two important needs that must be solved with auxiliary design documents.

The first use of auxiliary documentation is to capture important information from the problem space that did not make it directly into the design. Software design involves inventing software concepts to model concepts in a problem space. This process requires developing an understanding of the problem space concepts. Usually, this understanding will include information that does not directly end up being modeled in the software space, but which nevertheless helped the designer determine what the essential concepts were and how best to model them. This information should be captured somewhere in case the model needs to be changed at a later time.

The second important need for auxiliary documentation is to document those aspects of the design that are difficult to extract directly from the design itself. These can include both high-level and low-level aspects. Many of these aspects are best depicted graphically. This makes them hard to include as comments in the source code. This is not an argument for a graphical software design notation instead of a programming language. This is no different from the need for textual descriptions to accompany the graphical design documents of hardware disciplines.

Never forget that the source code determines what the actual design really is, not the auxiliary documentation. Ideally, software tools would be available that postprocessed a source-code design and generated the auxiliary documentation. That may be too much to expect. The next best thing might be some tools that let programmers (or technical writers) extract specific information from the source code that can then be documented in some other way. Undoubtedly, keeping such documentation up to date manually is difficult. This is another argument for the need for more expressive programming languages. It is also an argument for keeping such auxiliary documentation to a minimum and keeping it as informal as possible until as late in the project as possible. Again, we could use some better tools; otherwise we end up falling back on pencil, paper, and chalkboards.

To summarize,

- Real software runs on computers. It is a sequence of ones and zeros that is stored on some magnetic media. It is not a program listing in C++ (or any other programming language).
- A program listing is a document that represents a software design. Compilers and linkers actually build software designs.
- Real software is incredibly cheap to build, and it is getting cheaper all the time as computers get faster.
- Real software is incredibly expensive to design. This is true because software is incredibly complex and because practically all the steps of a software project are part of the design process.
- Programming is a design activity—a good software design process recognizes this and does not hesitate to code when coding makes sense.
- Coding actually makes sense more often than believed. Often, the process of rendering the design in code will reveal oversights and the need for additional design effort. The earlier this occurs, the better will be the design.
- Since software is so cheap to build, formal engineering validation methods are not of much use in real-world software development. It is easier and cheaper to just build the design and test it than to try to prove it.
- Testing and debugging are design activities—they are the software equivalent of the design validation and refinement processes of other engineering disciplines. A good software design process recognizes this and does not try to shortchange the steps.
- There are other design activities—call them top-level design, module design, structural design, architectural design, or whatever. A good software design process recognizes this and deliberately includes the steps.
- All design activities interact. A good software design process recognizes this and allows the design to change, sometimes radically, as various design steps reveal the need.
- Many different software design notations are potentially useful—as auxiliary documentation and as tools to help facilitate the design process. They are not a software design.
- Software development is still more a craft than an engineering discipline. This is primarily because of a lack of rigor in the critical processes of validating and improving a design.
- Ultimately, real advances in software development depend on advances in programming techniques, which in turn mean advances in programming languages. C++ is such an advance. It has exploded in popularity because it is a mainstream programming language that directly supports better software design.
- C++ is a step in the right direction, but still more advances are needed.

## Afterword

*As I look back on what I wrote almost 10 years ago, I am struck by several points. The first (and most relevant to this book) is that today I am even more convinced of the fundamental truth of the key points that I tried to make than I was then. My conviction is supported by a number of popular developments in the ensuing years that have reinforced many of the points. The most obvious (and perhaps least important) is the popularity of object-oriented programming languages. There are now many OO programming languages besides C++. In addition, there are*

*OO design notations such as the UML. My contention that OO programming languages have gained popularity because they allow more expressive designs to be captured directly in code seems rather passé now.*

*The concept of refactoring—restructuring a code base to make it more robust and reusable— also parallels my contention that all aspects of a design should be flexible and allowed to change as the design is validated. Refactoring simply provides a process and a set of guidelines on how to go about improving a design that has demonstrated some weaknesses.*

*Finally, there is the whole concept of Agile Development. While eXtreme Programming is the best known of these new approaches, they all have in common the recognition that the source code is the most important product of a software-development effort.*

*On the other hand, there are a number of points—some of which I touched on in the article—that have grown in importance to me in the ensuing years. The first is the importance of architecture, or top-level design. In the article, I made the point that architecture is just one part of design, and it needs to remain fluid as the build/test cycle validates the design. This is fundamentally true, but in retrospect, I think it was a little naïve of me. While the build/test cycle may reveal problems in an architecture, more problems are usually revealed by changing requirements. Designing software "in the large" is tough, and neither new programming languages, like Java or C++, nor graphical notations, such as UML, are of much help to people who do not know how to do it well. Furthermore, once a project has built a significant amount of code around an architecture, fundamentally changing that architecture is often tantamount to scrapping the project and starting over, which means it doesn't happen. Even projects and organizations that fundamentally accept the concept of refactoring are often still reluctant to tackle something that looks like a complete rewrite. This means that getting it right the first time (or at least close) is important, and getting more so as projects get larger. Fortunately, this is the area that software design patterns are helping to address.*

*One of the other areas that I feel needs more emphasis is auxiliary documentation, especially architecture documentation. While the source code may be the design, trying to figure out the architecture from the source code can be a daunting experience. In the article, I expressed the hope that software tools might emerge to help software developers automatically maintain auxiliary documentation from the source code. I have pretty much given up on that idea. A good object-oriented architecture can usually be described in a few diagrams and a few dozen pages of text. Those diagrams (and text) must concentrate on the key classes and relationships in the design, however. Unfortunately, I see no real hope that software tools are ever going to be smart enough to extract those important aspects from the mass of detail in the source code. That means people are going to have to write and maintain such documentation. I still think it is better to write it after the source code, or at least at the same time, than to try to write it before.*

*Finally, I remarked at the end of the article that C++ was an advance in programming—and hence software design—art, but that still more advances were needed. Given that I see a total lack of any real advances in programming art in the languages that have risen to challenge C++'s popularity, I feel this is even more true today than it was when I first wrote it.*

—Jack Reeves, January, 1, 2002

# Index


**A**

*A* metric, 265
Abbott, Edwin A., 177
Abstract, 317
  class, 264, 483
  Server, 317, 318
abstraction, 100, 194, 233, 249, 278
  metric *A*, 282
  underlying, 201
abusing patterns, 164
acceptance tests, 14
  framework, 27
activation rectangle, 485
Active Object, 151, 155, 500, 502
activity diagram, 496, 498
actors, 469
Acyclic Visitor, 107, 391
Acyclic-Dependencies Principle, 256, 260
Adapter, 138, 317, 319, 322, 325, 347, 360, 500
Adaptive Software Development, 19
ADP, 19
afferent couplings metric; *see* $C_a$ metric, 262
aggregation relationship, 480
Agile Alliance, 4
agile development, 87
analysis
  use case, 194, 201
analyst, 12
anticipation, 105
architecture, 8, 29, 199, 476
  system, 278
association relationship, 474, 480
asynchronization bar, 499
axis of change, 97

**B**

batch, 193
beauty, 32
Beck, Kent, vi, 4
Beedle, Mike, 4
Booch diagrams, 249
Booch, Grady, 444
bowling, 43
  rules, 83
Bridge, 317, 322, 325, 362, 365
Brittingham, Pete, 448
Brooks, Matt, 44, 55, 60, 61, 64
bubble sort, 165
bullpen, 15

**C**

C++, 133
$C_a$ metric, 262
card, index, 12, 20
CASE, 5
$C_e$ metric, 262
change, isolation of, 278
charts, big visible, 15
closure, 201, 233, 249
cloud icon, 474
Cockburn, Alistair, 1, 4
code
  website, 250
  duplication, 16
  rot, 16, 87
cohesion, 95, 253, 256, 281
collaboration diagram, 503
Collective ownership, 14
Command, 151, 195, 205, 212, 294, 500
Common-Closure Principle, 256, 260, 261, 277, 329, 404, 418
Common-Reuse Principle, 255, 260, 279
communication, 7
comp.object, 317

Component, 103
complex transitions, 499
Composite, 293
composition, 198
  relationship, 479, 480
compromise, 122
continuous integration, 14
contract, 5
conversation, 7
copy
  the Copy program, 90, 127
corporations, 4
coupling, 253, 256, 281
CppUnit, 348
<<creates>> stereotype, 486
Craig, Philip, 29
Crystal, 19
Cunningham, Ward, vi, 4, 24
customer, 5, 12, 15, 20
  tests, 27

**D**

*D* metric, 267
*D′* metric, 267
data token, 485
database, 5, 15, 21, 194, 195, 207, 248, 328, 353
  *...an implementation detail*, 194
database interface layer, 481
Date class, 236
Decorator, 354, 403
decoupling, 24, 27, 29
  physical, 154
  temporal, 154
DeMarco, Tom, 95, 281
dependency
  inversion, 94, 98, 278
  management, 94, 98, 233, 253
  management metrics, 281
  relationship, 476
  structure, 127
Dependency-Inversion Principle, 127, 161, 171, 259, 263, 265, 269, 274, 278, 282, 316, 318, 320, 329, 347, 368, 418, 456
design, 87
  by contract, 117
  principles, 86
  quick~session, 25
  rot, 90
  smell, 86, 88, 103
  simplicity, 15
  Test-First, 14
Development, Test-Driven, 14
DIP, *see* Dependency-Inversion Principle, 127
directed acylic graph, 258
DLL, 103
documentation, 5, 25, 27
domain model, 472
DTSTTCPW, 16
dump trucks, 17
duplication, code, 16
`dynamic_cast`, 214

**E**

Educational Testing Service, *see* ETS, 443
efferent couplings metric, *see* $C_e$ metric 262
Eiffel, 117
EJB, 16
emergent design, 8
Equivalence, 278
Eratosthenes, 32
estimate, 12
estimation, 20
ETS, 443, 447
experience, 202
exploration, 20
<<extends>> relationship for use cases, 470
extension
  object, 353, 408
  point, 469
Extreme programming, vi, 11, 19, 194

**F**

Facade, 98, 173, 179, 207, 354
factoring, 122, 123
Factory, 269, 285, 363, 376, 417
fat interfaces, 135
FDD, 19
feature-driven development, 19
feedback, 23
final, 493
  pseudostate, 493
Finite state
  automata, 419
  machine, 182, 419, 458, 465, 492
Flatland, 177
flowchart, 498
Fowler, Martin, 4, 31
fragility, 88, 96, 98, 103, 104, 116, 140
Framework, 445
Freeman, Steve, 29
FSM, *see* finite state machine, 419
<<function>> stereotype, 486
functional
  decomposition, 260
  tests, 27
furnace, 132

**G**

Galactic Modeling Language, 490
game, planning, 15, 19
Gantt, 6
garbage, 17
<<generates>> stereotype, 477
global variables, 143
GML, 490
graph, 258
Grenning, James, 4
GUI, 21

**H**

*H* metric, 282
Highsmith, Jim, 4

hooks, 105
Hunt, Andy, 4, 56

I
*I* metric, 262
Immobility, 88, 103, 104
<<include>> relationship for use cases, 470
index card, 20
infrastructure, 7, 15, 208
inheritance, 111, 161
  public, 121
initial, 493
  pseudostate, 493
instability metric, 262
instances, 274
integration, continuous, 14
intelligence, 202
IntelliJ, 34
intentional programming, 24, 27
interface pollution, 136
Interface-Segregation Principle, 135, 137, 316, 320, 324
internal transition, 493
interrupt, 490
  service routine, 491
invariant, 114
inversion, 127
ISA, 113, 125
ISP, *see* Interface Segregation Principle, 135
ISR, 491
iteration, 12, 21, 22, 106
iterative development, 106

J
Jacobson, Ivar, 99, 194
JDBC, 328
Jeffries, Ron, 14
judgment, 122, 279

K
Kern, Jon, 4
Khrushchev, Nikita, 317
kludge, 98
Koss, Bob, 43

L
lifeline, 484
link relationship, 480
link-time polymorphism, 486
Linux, 236
Liskov, Barbara, 111
Liskov-Substitution Principle, 111, 136, 418, 316, 320, 321, 322
loop box, 485
LSP, *see* Liskov-Substitution Principle, 111

M
main sequence, 265, 266, 281
  conformance, 281
  distance metric *D*, 282
  normalized distance metric *D′*, 282
Mackinnon, Tim, 29
marathon, 15
Marick, Brian, 4
marketing, 12
Martin, Robert C., 4, 241
Martin's first law of documentation, 472
Mediator, 173
Mellor, Steve, 4
merge, 14
message, 484
  sending to self, 209
message sequence chart, 505
Metaphor, 16
metric
  abstractness *A*, 282
  afferent coupling $C_a$, 282
  distance *D*, 282
  efferent coupling $C_e$, 282
  instability *I*, 282
  normalized distance *D′*, 282
  relational cohesion *H*, 282
  stability, 281
metrics stability, 262
Meyer, Bertrand, 99, 117
MIT Sloan Management Review, 6
Mock Object, 299
modem, 97
Monostate, 177, 180, 213
morning-after syndrome, 256, 260
multiple inheritance, 308
multiplicity, 474, 475
multithreading, 16, 155

N
names, system of, 17
National Council of Architectural Registration Boards, *see* NCARB, 443
natural structure, 105
NCARB, 443
Needless
  Complexity, 86, 88, 97, 105, 116, 136, 213, 339
  Redundancy, 136
  Repetition, 88
nondeterministic behavior, 159
Norman, Donald A., 275
Null Object, 189, 199, 202, 219

O
OAOO, 16
object
  diagram, 500, 501
  factory, 285
  icon, 484
Object Mentor Inc., 82, 444
object-oriented
  database, 248
  design, 7
Observer, 297, 315, 359, 360, 370
OCP, *see* Open-Closed Principle, 99
office, open, 15
oma.com, 236
OMI, *see* Object Mentor Inc., 444
OODBMS, 248

Opacity, 88
open workspace, 15
Open-Closed Principle, 86, 93, 99, 111, 114, 199, 201, 233, 249, 256, 264, 295, 316, 318, 320, 321, 322, 359, 360, 418
ORB, 16
overtime, 15

**P**

pace, sustainable, 15
package, 481
  principles, 253
Pair Programming, 13, 43
Page-Jones, Meiller, 95
parameter stereotype, 486
pattern, 7
  abuse, 164
Pattern: Mock Object, 25
Payroll, 25, 193
persistence, 98, 248
PERT, 6
petri net, 498
plan, 6
  release, 12
planning, 15
  game, 15, 19
point, 177
  story, 20
polymorphism, 113, 233, 249
  link-time, 486
postcondition, 117
pragmatic programmers, 56
precondition, 117
prescience, 105
principles, 7
principles of object-oriented design, 14
procedural programming, 101
process, 3
programming by coincidence, 56
Proxy, 98, 272, 274, 294, 327, 329, 374
pseudostate, 493
  final, 493
  initial, 493
pure virtual function, 282

**Q**

quality, 8
quality assurance, 27
QuickEntryMediator, 174

**R**

race condition, 505
RDBMS, 249
real world modeling, 105
realizes relationship, 483
reentrancy, 491
Reeves, Jack, 87, 517
refactoring, 16, 31, 43
  browser, 34
relational
  cohesion metric, 282
  database, 194, 249
release, 12
  structure, 279
release plan, 20
requirements, 12
reusable framework, 445
reuse, 445, 447
reuse–release
  equivalence, 254
  equivalence principle, 278
Rigidity, 86, 88, 97, 98, 99, 103, 104, 137, 140
ring buffer, 491
RMI, 16
rot, 87, 90
RRCJ3, 241
RTC, *see* run-to-completion, 158
rude, 52
Rufus, 507
run-time type information, 112
run-to-completion, 158
Rupert, 507

**S**

Schwaber, Ken, 4
SCRUM, 19
self-organization, 8
separation of concerns, 345
sequence diagram, 484
shakeholders, 472
shared library, 103
short cycles, 106
sieve, 32
simple design, 15
simplest thing that could possibly work, 16
simplicity, 8, 15, 297
Single-Responsibility Principle, 52, 75, 95, 195, 201, 256, 260, 319, 329, 361, 374, 396, 404, 408, 418
Singleton, 177, 178, 213, 486
SMC, 429, 461, 462, 464
software rot, 87
source code control system, 14, 89
source file organization, 275
sparcstation, 236
specialties, 14
spike, 20
spoofing, 273
SQL, 329
SRP, *see* Single-Responsibility Principle, 95
stability, 256, 261, 262, 281
Stable-Abstractions Principle, 264, 278
Stable-Dependencies Principle, 261, 263, 265
Stairway to Heaven, 327, 347
stake holders, 106
State, 419, 426, 461, 465, 493
  transition diagram, 419
  transition table, 420
static polymorphism, 133
STD, *see* state transition diagram, 419
stereotype, 461, 473
story points, 20
strategic closure, 105
Strategy, 94, 101, 161, 168, 198, 202, 237, 428, 456

structured
  analysis, 127
  design, 127
STT, *see* state transition table, 420
substate, 494
subsystem, 482
superstate, 494
Sutherland, Jeff, 4
Sutter, Herb, 215
switch statement, 103
system
  architecture, 278
  of names, 17

**T**

table lamp problem, 317
task, 12, 14, 21
  sign-up, 21
Taskmaster, 464
team, 4
Template Method, 101, 161, 209, 220, 222, 223, 224, 226, 227, 229, 230, 350, 361, 403, 450, 453, 454, 455, 456
templates, 133
temporal, 154
Tennyson, Alfred, 189
test-driven development, vi, 43, 98, 106, 116, 272
test-first design, 6, 14
Tests, Acceptance, 13
Thomas, Dave, 4
Thomas, Pragmatic Dave, 56
thread, 496
tools, 4
top-down design, 260
transactions, 153
transition; internal, 493
transitions between states, 494
type stereotype, 472

**U**

UML, 15, 25, 45, 87, 193, 205, 219, 230, 242, 253, 467, 489
Uncle Bob, 77
underlying abstraction, 201
UNDO, 154
unit test, 23
use cases, 194, 202, 469
  analysis, 201
user
  stories, 12, 193
  story, 20, 22
    merging, 20
    splitting, 20

**V**

van Bennekum, Arie, 4
velocity, 20, 21, 22
Viscosity, 88, 137, 140
Visitor; acyclic, *see* Acyclic Visitor, 233, 353, 387, 388
volatility, 130

**W**

war room, 15
website, 250
weekly build, 257
Wiener, Lauren, 123
wiki-wiki-web, vi
Wilkerson, Brian, 123
Williams, Laurie, 13
Windows 2000, 236
Wirfs-Brock, Rebecca, 123, 446
Woolf, Bobby, 190
workspace, open, 15
Wumpus, 24

**X**

XML, 28
XUnit, 348

**Y**

YAGNI, 16

# Practices of eXtreme Programming

## Whole Team

All the contributors to an XP project—developers, business analysts, testers, etc.—work together in an open space, members of one team. The walls of this space are littered with big visible charts and other evidences of their progress.

## Planning Game

Planning is continuous and progressive. Every two weeks, for the next two weeks, developers estimate the cost of the candidate features, and customers select those features to be implemented based upon cost and business value.

## Customer Tests

As part of selecting each desired feature, the customers define automated acceptance tests to show that the feature is working.

## Simple Design

The team keeps the design exactly suited for the current functionality of the system. It passes all the tests, contains no duplication, expresses everything the authors want expressed, and contains as little code as possible.

## Pair Programming

All production software is built by two programmers, sitting side by side, at the same machine.

## Test-Driven Development

The programmers work in very short cycles, adding a failing test, then making it work.

## Design Improvement

Don't let the sun set on bad code. Keep the code as clean and expressive as possible.

## Continuous Integration

The team keeps the system fully integrated at all times.

## Collective Code Ownership

Any pair of programmers can improve any code at any time.

## Coding Standard

All the code in the system looks as if it was written by a single—very competent—individual.

## Metaphor

The team develops a common vision of how the program works.

## Sustainable Pace

The team is in it for the long term. They work hard, at a pace that can be sustained indefinitely. They conserve their energy, treating the project as a marathon rather than a sprint.

# The Principles of Object-Oriented Design

**SRP** **The Single Responsibility Principle**

*A class should have only one reason to change.*

**OCP** **The Open-Closed Principle**

*Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.*

**LSP** **The Liskov Substitution Principle**

*Subtypes must be substitutable for their base types.*

**DIP** **The Dependency Inversion Principle**

*Abstractions should not depend upon details. Details should depend upon abstractions.*

**ISP** **The Interface Segregation Principle**

*Clients should not be forced to depend upon methods that they do not use. Interfaces belong to clients, not to hierarchies.*

**REP** **The Release-Reuse Equivalency Principle**

*The granule of reuse is the granule of release.*

**CCP** **The Common Closure Principle**

*The classes in a package should be closed together against the same kinds of changes. A change that affects a closed package affects all the classes in that package and no other packages.*

**CRP** **The Common Reuse Principle**

*The classes in a package are reused together. If you reuse one of the classes in a package, you reuse them all.*

**ADP** **The Acyclic Dependencies Principle**

*Allow no cycles in the package dependency graph.*

**SDP** **The Stable Dependencies Principle**

*Depend in the direction of stability.*

**SAP** **The Stable Abstractions Principle**

*A package should be as abstract as it is stable.*

# AGILE SOFTWARE DEVELOPMENT

## *Principles, Patterns, and Practices*


Robert C. Martin

with contributions by James W. Newkirk and Robert S. Koss


Best-selling author and world-renowned software development expert Robert C. Martin shows how to solve the most challenging problems facing software developers, project managers, and software project leaders today. This comprehensive, pragmatic tutorial on Agile Development and eXtreme Programming, written by one of the founding fathers of Agile Development:

- Teaches software developers and project managers how to get projects done on time, and on budget using the power of Agile Development.
- Uses real-world case studies to show how to plan, test, refactor, and pair program using eXtreme Programming.
- Contains a wealth of reusable C++ and Java™ code.
- Focuses on solving customer-oriented systems problems using UML and Design Patterns.

### About the Author

**Robert C. Martin** is President of Object Mentor Inc. Martin and his team of software consultants use Object-Oriented Design, Patterns, UML, Agile Methodologies, and eXtreme Programming with worldwide clients. He is the author of several best-selling books, including *Designing Object-Oriented C++ Applications Using the Booch Method* (Prentice Hall, 1995). Dr. Martin was also chief editor of *Pattern Languages of Program Design 3* (Addison Wesley, 1997), editor of, *More C++ Gems* (Cambridge, 1999), and co-author of *XP in Practice*, with James Newkirk (Addison-Wesley, 2001). He was Editor-in-Chief of the *C++ Report* from 1996 to 1999 and is a featured speaker at international conferences and trade shows.

*"This is perhaps the first book to weave agile methods, patterns, and the fundamentals of modern software development into a coherent whole. When Bob Martin speaks, better listen up."*

**—John Vlissides, consultant and author of *Design Patterns and Pattern Hatching***

*"I've been waiting for this book for far too long, Bob has an enormous amount to teach about the pragmatics of mastering our craft."*

**—Martin Fowler, consultant and author of *UML Distilled and Refactoring***

*"In* Agile Software Development: Principles, Patterns, and Practices, *Bob Martin demonstrates his gift as both master developer and educator. He enlightens us with his practical insight and delightful style."*

**—Craig Larman, consultant and author of *Applying UML and Patterns***


ISBN 0-13-597444-5

90000

9 780135 974445

Pearson Education

Prentice Hall
Upper Saddle River, NJ 07458
www.prenhall.com